

Chapter 2. Markov Decision Processes, Dynamic Programming, and Monte Carlo Methods

The foundation of reinforcement learning (RL) is based upon three topics. The most important is the *Markov decision process* (MDP), a framework that helps you describe your problem. *Dynamic programming* (DP) and *Monte Carlo* methods lie at the heart of all algorithms that intend to solve MDPs. But first, let me discuss an application that you have probably heard of, but didn't consider to be RL.

Multi-Arm Bandit Testing

Imagine you work for an online ecommerce company. Given your role in engineering, you will be expected to develop new features and maintain existing ones. For example, you might be asked to improve the checkout process or migrate to a new library.

But how can you be certain that your changes have the desired effect? Monitoring key performance indicators (KPIs) is one possible solution. For new features you want to positively impact the KPIs. For maintenance tasks you want no impact.

As an example, take the classic scenario of establishing the best color for a button. Which is better, red or green? How do you quantify the difference? To approach this using RL you must define the three core elements of the problem: the reward, the actions, and the environment.

TIP

In-depth practical advice on the development of RL solutions is provided in [Chapter 9](#).

Reward Engineering

To quantify performance, the result of an action must be measurable. In RL, this is the purpose of the reward. It provides evidence of an outcome, which can be good or bad. Practitioners typically encode good outcomes as positive values (but this is not strictly necessary).

What reward signal should you choose for the website button example? You could say that clicks are a positive outcome. And a sale is a strongly positive outcome. Often you will have limited access to metrics and have to compromise. For example, rather than use the sale as the KPI, you could calculate the average sale for everyone who clicks the button and assign that single value to every click. That way you would not need to integrate with another system.

For the related problem of testing that new software doesn't have a significant detrimental effect, you might be able to get away with something as simple as button clicks with a simple +1 reward. If the metrics show a similar number of clicks then you can be happy you haven't broken anything—a successful day of engineering.

Either way, the reward must be quantitative and, ideally, it should be as simple as possible. If you use an overly complex reward signal then the proposed algorithms might overfit and learn to take advantage of a flaw in the reward signal, rather than solving the intended problem. For example, distance-to-goal-based rewards are often used and are usually quite robust, but an agent trying to navigate a maze using this reward is likely to get stuck in a dead end near the goal, because being close to the goal provides *nearly* optimal rewards.

Next, how can you use the reward to quantify the best outcome?

Policy Evaluation: The Value Function

In an environment, an agent takes a specific action, a , which is a member of the mathematical set $\mathcal{A}$. In other words, the agent could have taken one of many actions from the set $\mathcal{A}$, but on this occasion it chose a . This is written as $a \in \mathcal{A}$.

The environment rewarded the agent with a specific reward, r , which comes from a wide range of potential rewards defined by the set $\mathcal{R}$. This moved the environment into a new state, $s \in \mathcal{S}$.

All of these variables could be stochastic, meaning that if your agent requested exactly the same action in precisely the same situation, the environment may return a slightly different reward or state. In the literature you will see these denoted as uppercase letters, but in this book I only use the lowercase terminology to simplify the notation, even though it might not be strictly accurate.

In the previous website button example, the action is to choose whether to show the red button or the green button to a visitor, which in mathematical form would be $\mathcal{A} \doteq \{a_{red}, a_{green}\}$. Whenever a user visits your website your agent will decide which action to choose, as defined by the current strategy. The user then clicks the button or doesn't and a reward is sent back to your agent. It's easiest to imagine the reward being one for a click and zero for no click, for example. But it could easily be the profit made from that customer or any other business metric.

So back to the original question, which is best? One way of quantifying performance is to calculate the average reward, r^{avg} , for a certain number of customers, N , for each action, a , shown in [Equation 2-1](#).

NOTE

Remember that presenting a single button and waiting for a single click is not a sequential task; there is only a single decision. This is the primary difference between bandits and RL. I will demonstrate sequential decisions shortly.

Equation 2-1. Average reward calculation

$$r^{avg}(a) \doteq \frac{1}{N(a)} \sum_{i=1}^{N(a)} r(a)_i = \frac{r_1 + r_2 + \dots + r_{N(a)}}{N(a)}$$

[Equation 2-1](#) explains that to calculate the average reward for each action, you sum the observed rewards and divide by the number of customers.

This works, but is not computationally efficient in terms of storage. More importantly, you'd have to wait until lots of customers have interacted with your button before you can compute this.

Instead you can reformulate [Equation 2-1](#) as an *online* algorithm. This means that the parameters of the algorithm are updated in place, rather than waiting until the end of all the experiments. [Equation 2-2](#) derives this explicitly, because it is important for many subsequent RL algorithms.

Equation 2-2. Online value function

$$r_N^{avg} \leftarrow \frac{1}{N} \sum_{i=1}^N r_i \quad (1)$$

$$\leftarrow \frac{1}{N} \left(r_N + \sum_{i=1}^{N-1} r_i \right) \quad (2)$$

$$\leftarrow \frac{1}{N} \left(r_N + (N-1) \frac{1}{N-1} \sum_{i=1}^{N-1} r_i \right) \quad (3)$$

$$\leftarrow \frac{1}{N} (r_N + (N-1)r_{N-1}^{avg}) \quad (4)$$

$$\leftarrow \frac{1}{N} (r_N + Nr_{N-1}^{avg} - r_{N-1}^{avg}) \quad (5)$$

$$\leftarrow r_{N-1}^{avg} + \frac{1}{N} (r_N - r_{N-1}^{avg}) \quad (6)$$

This is one of those mathematical manipulations that is only obvious in hindsight. The goal is to convert a static average calculation into an on-line version using mathematical tricks. If you do not concentrate, the tricks are easy to miss.

I begin in step (1) by reusing [Equation 2-1](#), but this time I have removed the notation stating that the reward is a function of the action. Remember that this is the same for each action. I also use the assignment operator, $\leftarrow$, to denote that I am updating the variable in place. It is not an equality.

In step (2) I start to extract the most recent reward from the sum, so that the summation operator ends at the second-to-last customer, $N-1$.

In (3), I use my mathematical trickery to multiply the sum by $1 = (N-1)/(N-1)$. This move requires the foresight to see that $\frac{1}{N-1} \sum_{i=1}^{N-1} r_i$ is in the same form as step (1), which makes it equivalent to r_{N-1}^{avg} . Hence, in step (4), I can replace the expansion from the previous step with r_{N-1}^{avg} .

Between (4) and (5) I perform some multiplication to expand the $(N - 1)$ and you can see that there is a $1/N$ and an N that cancel each other out.

In (6), after multiplying by $1/N$, the result looks like an exponentially weighted moving average (also known as exponential smoothing), a common form of online algorithm to estimate the mean.

In fact, most RL algorithms operate in this way. You can clean up the notation in [Equation 2-2](#) to result in the common form of the exponentially weighted moving average algorithm, shown in [Equation 2-3](#).

Equation 2-3. General form of an exponentially weighted moving average

$$r = r + \alpha(r' - r)$$

This might sound familiar. This is the same as [Equation 1-3](#), except here I have not used matrix mathematics. Calculating the error and updating the predictions accordingly is a common theme across machine learning and RL.

Now that you know how to calculate the reward and how to update it online, the next challenge is to decide which action to take, based upon the these average rewards.

Policy Improvement: Choosing the Best Action

Allowing an agent full control over the choices it makes distinguishes RL from ML. You need to decide how the agent makes those decisions. Let me start with the simplest solution first, then expand from there.

Using the same website button example, assume that you have already decided that the agent receives a fixed reward for every button click.

You could show the new version of the website 50% of the time. Later, you can sum the reward (the number of clicks) to find the preferred button. This is precisely what A/B testing does. But this solution has a major flaw: 50% of customers see a sub-optimal button. Your company loses sales.

In a second approach, you could show the new version 50% of the time to begin with, but shift the split toward the preferred option over time. This is the idea behind bandit algorithms.

This strategy highlights an important problem in RL. You need to observe the rewards from all of the actions multiple times before you know which is best. But you also want the agent to exploit the best known actions to maximize your reward now. This is the compromise between *exploration* and *exploitation*.

A generalized version of this algorithm devotes some time to exploration and some to exploitation. Upon every action there is some probability that the action will be a random choice, specified by the parameter ϵ . This is called the ϵ -greedy algorithm and is shown in [Algorithm 2-1](#). r^{avg} and N have the same meaning as before.

Algorithm 2-1. ϵ -greedy bandit algorithm

- 1: **input:** a exploration probability $0 \leq \epsilon \leq 1$,
- 2: Initialize $r^{avg}(a) \leftarrow 0, N(a) \leftarrow 0$, for each $a \in \mathcal{A}$
- 3: **loop** for ever:
 - 4: $a \leftarrow \begin{array}{ll} \text{argmax } r^{avg}(a_s) & \text{with probability } 1 - \epsilon, \text{ breaking ties randomly} \\ \text{random } a & \text{with probability } \epsilon \end{array}$
 - 5: Present action a in the environment and receive reward r
 - 6: $N(a) \leftarrow N(a) + 1$
 - 7: $r^{avg}(a) \leftarrow r^{avg}(a) + \frac{1}{N(a)}[r - r^{avg}(a)]$

Step (1) states that you need to pass in a value for ϵ between zero and one. Higher values will explore. Lower values will exploit. Step (2) creates arrays to store the results. Step (3) tells you to loop forever.

The algorithm gets interesting in step (4). With a probability of $1 - \epsilon$, the agent should choose the action that produces the current highest average reward. Similarly, the agent should select a random action with probability ϵ .

Step (5) runs the chosen action in the environment and returns the reward, which I define shortly. For example, if the algorithm chose the red button, your UI should present that and inform you if the user clicked. This is the interaction with the environment.

Step (6) increments a counter for this action for use next. And finally, step (7) updates the online average reward function ([Equation 2-2](#)) based upon the instantaneous reward, the action, and the number of times the agent has chosen that action. Over time this value estimate will become more accurate, assuming the rewards don't change.

[Algorithm 2-1](#) is a bandit algorithm, named after the one-armed slot machines. Over time this algorithm allows you to automatically pick the “arm” on the bandit machine that provides the highest reward. If you wanted to perform automated bandit testing, a process of comparing two or more competing solutions, [Algorithm 2-1](#) allows you to automatically tend toward the version that produces the highest reward.

Simulating the Environment

When you work on an industrial project, the best environment is real life. However, your environment may be too expensive, dangerous, or complicated to develop against. Instead, you can build a simulation, which could be a simplified version of a domain (like a three-dimensional gaming engine), or modeled from collected data.

Pretend that the notional website visitors do not click the button every time, only some proportion of the time. Also, they prefer the red button over the green button. In this example, I assume that a whopping 40% of users click the red button, but only 5% of users click the green. But these numbers could come from prior experimentation or a published model. You can use this information to create a simple simulator to test your RL algorithm, as shown in [Algorithm 2-2](#).

Algorithm 2-2. Website simulator

```
function ENVIRONMENT( $a, p(a)$ )  
input: The action,  $a$ , and its probability,  $p(a)$   
output:  $r \leftarrow 1$  with probability  $p(a)$ , else 0.
```

I know this is simple, but I thought it was useful to spell out that simulations don't have to be complex to be useful. It accepts an action and the probability of an action and outputs a reward of 1 with probability $p(a)$. I provide a reward of 1 here, but you could return a number that makes more sense in your context. For example, the total value of the sale (in which case you will need to simulate order values as well).

Running the Experiment

Now you can run your first experiment. [Figure 2-1](#) illustrates the workflow of a typical bandit test on a website. This image might differ from what you expect, because I explicitly include the following RL components: the agent, the action, and the reward. First the user browses to your URL and the agent decides which button to show. Then the user views the page and may click the button. The result passes back to the agent so it can learn which action to choose in the future.

NOTE

All code is available for you to use online on the [accompanying website](#) (see [“Supplementary Materials”](#) for more information).

I implemented [Algorithm 2-2](#) using two Bernoulli distributions, which represent the two buttons, with the probability of being selected set to 40% for the red button and 5% for the green. I then implemented [Algorithm 2-1](#) to learn which button was preferred by these simulated users automatically.

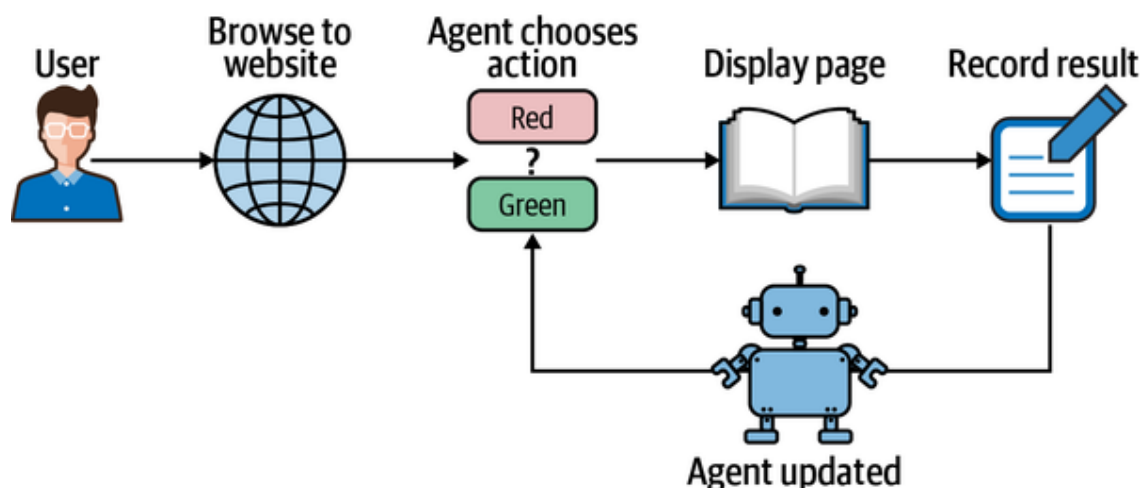


Figure 2-1. A depiction of the bandit testing workflow.

[Figure 2-2](#) compares three different values of the hyperparameter ϵ , which I pass into [Algorithm 2-1](#). Each simulation ran for 250 steps, which means that the algorithm had 250 opportunities to learn the underlying button-color preferences (that is, the simulated customer prefers the red button) and extract the most value. Each line represents the percentage of time that the agent chose the optimal button to show to the visitor, on average, over 1,000 repeats to reduce the noise due to random sampling.

Looking at the start of the experiment, you can see that it took some time for each run to learn the optimal button. The rate of this learning depends on the following factors. When ϵ is high, the agent chooses random actions most of the time and the agent has many opportunities to stumble across the best action. When ϵ is low, the agent has fewer opportunities to explore and you will have to wait for a long time. In other words, ϵ affects the learning speed.

Looking toward the end of the experiment, different values of ϵ affect the eventual optimal reward percentage. I say eventual because each curve is asymptotic, which means tending toward some final value. Imagine you are the algorithm with an ϵ of 1. This means that you always select a random action (show a random button). This means the maximum reward you could ever get is 50%. If you start to reduce ϵ , then you do get to choose the optimal action, some of the time. The asymptote of the optimal action is $\epsilon/n + (1 - \epsilon)$, where n is the number of actions. When $\epsilon = 1$, the agent always picks a random action and the result is an optimal action of 0.5. When $\epsilon = 0.1$, the result is an optimal action of 0.95. ϵ affects the final rate at which the algorithm chooses the optimal action.

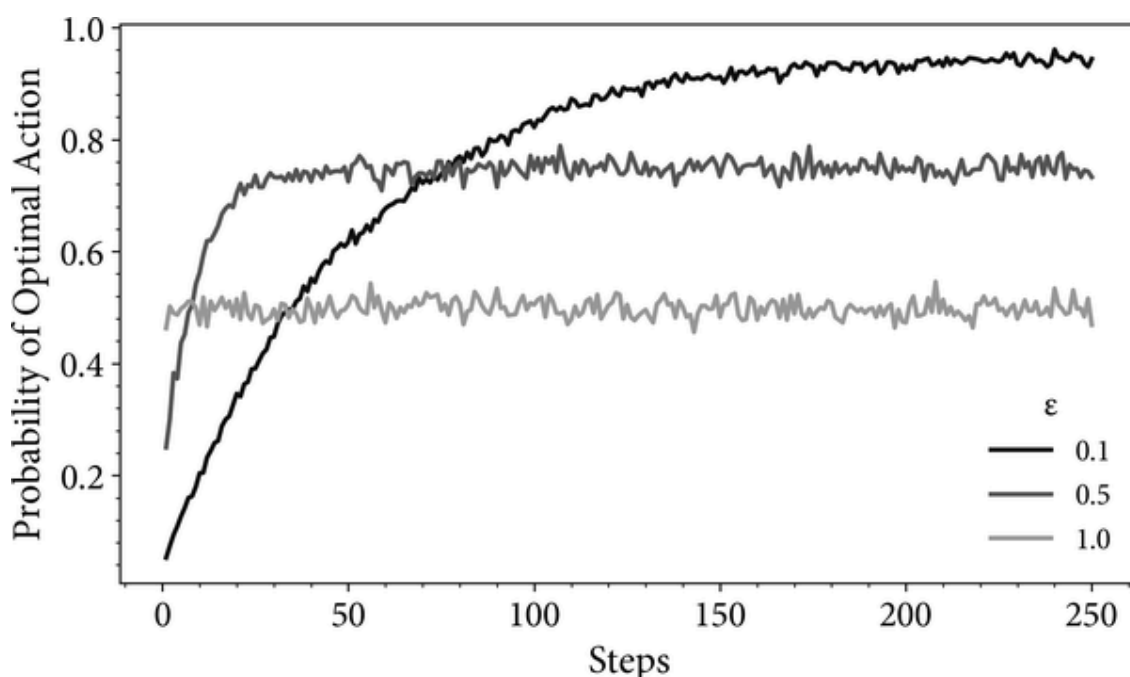


Figure 2-2. The effect of altering the hyperparameter ϵ , the proportion of the time you want the agent to explore. High values of ϵ lead to greater exploration, but less exploitation of the optimal action and vice versa.

Improving the ϵ -greedy Algorithm

All algorithms are fundamentally limited by how effectively they explore and how quickly they exploit. The best algorithms are able to do both at

the same time, but defining how to explore is largely problem dependent. For example, a maze and an optimal healthcare schedule would require different exploration strategies.

The previous section showed that the hyperparameter ϵ controls this trade-off. Too high and the agent does too much searching, which results in choosing bad actions. Too low and the agent does too little searching and takes a long time to find new optimal actions.

TIP

I want provide some intuition on other simple exploration methods here. See the literature in [“Further Reading”](#) for more details.

The simulation also factors into this discussion. If the difference between the rewards of two actions is small, the agent has to sample these two outcomes a lot. According to the law of large numbers, the agent’s confidence bound on the reward becomes smaller with more observations.

It is often better to choose the action based upon the current estimates of the distribution of rewards. In other words, rather than returning a single action, the agent returns the probabilities of each action, weighted by expected rewards of each state. This is called a *softmax* function. This provides a natural exploration function defined by the current reward estimates.

If the eventual aim is to extract as much reward as possible, then there is no point continuing to explore. You could remove the ϵ -greedy action. But it is more common to reduce the value of ϵ over time. This is called *annealing*. The word annealing comes from metallurgy. It means to heat and cool metals slowly, to strengthen and remove stresses.

A third popular improvement revolves around the idea that it doesn’t make sense to explore randomly, especially in simple simulations. The agent will learn more by exploring states that it hasn’t seen before. These algorithms add a *bonus* to each action for inadequately sampled states and are called *upper-confidence-bound* (UCB) methods. UCB algorithms are useful because they have no hyperparameters like ϵ (or the rate at which ϵ decreases, for annealing).

[Figure 2-3](#) compares these three methods. I chose the value of ϵ to approximate the same final performance as the other methods. You can see that the annealing-softmax and the UCB strategies learn an optimal action far quicker than the ϵ -greedy method. UCB also benefits from having no tunable parameters.

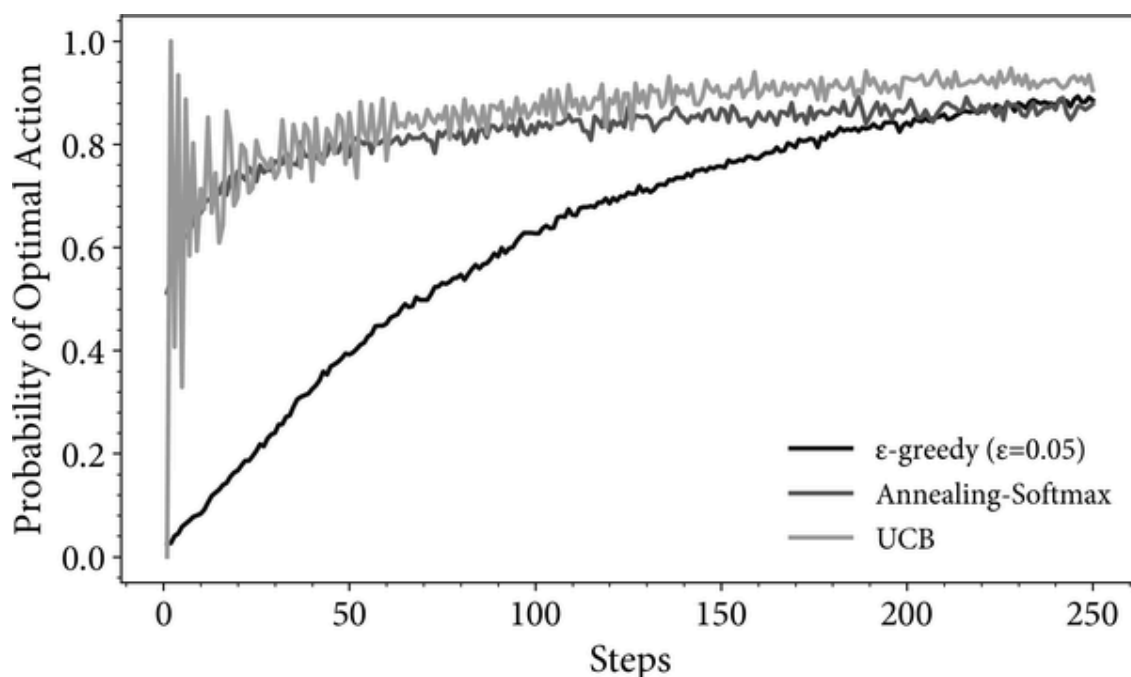


Figure 2-3. A comparison of three common exploration strategies: ϵ -greedy, annealing-softmax, and UCB.

TIP

Increasing the rate at which algorithms learn is a common goal of new RL algorithms.

Markov Decision Processes

A Markov decision process (MDP) is a mathematical formalism of many of the ideas introduced in [Chapter 1](#). An *agent* influences the observed behavior of a stochastic system (the *environment*) by choosing *actions*. The goal is to choose the set of actions that allow the system to behave in an optimal way, as defined by some success criterion (the *reward*).

The agent can never fully appreciate the environment. The agent receives a limited set of *observations* that represent the current *state* of the system. These two words mean exactly what they are. The state is the condition that the environment is in at a particular time. The observation is the

occluded view of that state. In many environments the state can be directly observed.

Digital agents perform actions at discrete times. This gives rise to the idea of a *step* in time. Note that this does not prevent the use of continuous actions, like the angle of a steering wheel. The opposite of a digital agent, an analog agent, much like a mechanically damped system, is technically possible but I have never seen one. An interesting hobby project perhaps?

As such, the agent and environment interact in a sequence of discrete time steps, t . At each time step the agent receives a representation of the state, s . Based upon the state of the environment the agent can suggest an action, a , and one time step later receives a value representing the reward, r , and a new set of states, s' . Here, the apostrophe above the s is used to represent the *next* state. You will often see subscripts with t and $t + 1$ that mean the same thing. [Figure 2-4](#) visualizes these elements.

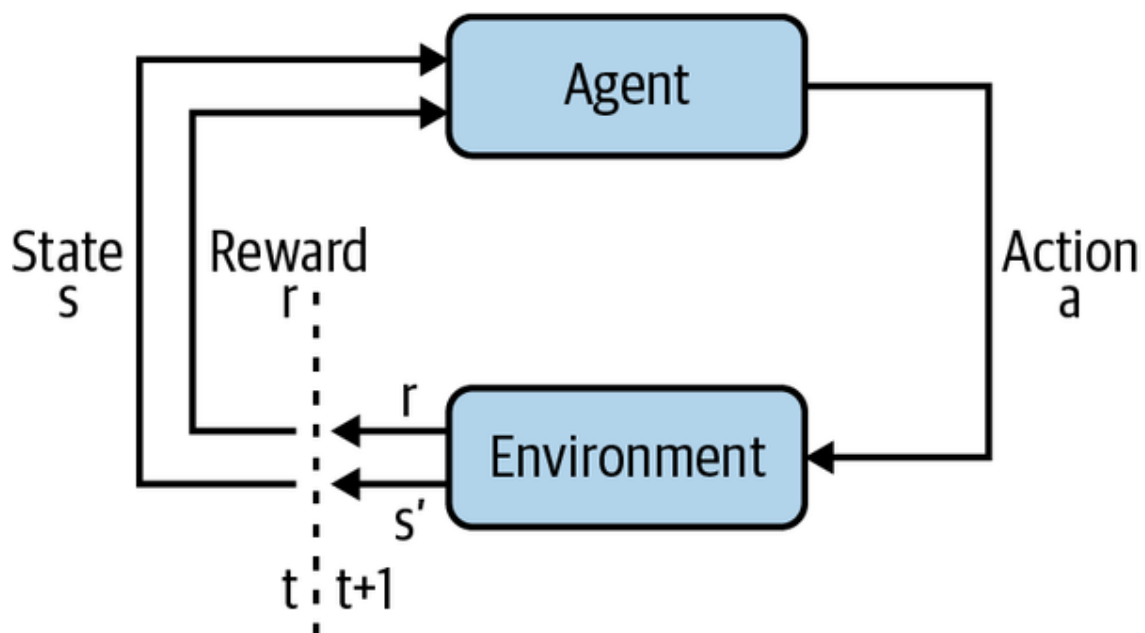


Figure 2-4. A representation of the interface and feedback loop between the environment and the agent in an MDP. The agent accepts the current state and reward from the environment and based upon this information performs an action. The environment takes the action and produces a new reward and state for the next time step.

Remember that the states and rewards are random variables, not definite values, because most environments are stochastic. For example, your agent could choose to turn the steering wheel in a self-driving car, but there is no guarantee that the car will actually turn; it might be on ice. To cater for this, the concept of a *transition model* is used. This is also called the *dynamics* of the MDP in some books. The transition model, shown in [Equation 2-4](#), is the probability of landing in a new state with a reward

given a previous state and performed action. The notation has been simplified to emphasize the causal effect of the action.

Equation 2-4. Transition model of an MDP

$$p(s', r | s, a)$$

[Equation 2-4](#) is the key to the MDP. In a Markov decision process, the transition model describes the dynamics of the environment. The next state and reward depend only on the previous state and the action. And the only difference between an MDP and a traditional Markov chain, which is also called a *Markov process*, is the addition of the action and reward parameters. The action is a *decision*, which is the “D” in MDP.

TIP

I like to think of the MDP as an interface. It is a contract between the agent and the environment. The agent can only observe the states and suggest an action. In return the environment will give you a new state and some value that represents how well you are doing. This might sound constrictive, but it allows for a remarkable amount of flexibility on both sides of the interface. The environment can be as complex as it needs to be. It could be a simulation of users clicking a button, an Atari game, or even real life. The agent also can be as complex as it needs to be. It could be deciding to show a green or red button, whether to move left or right in a game, or if it is a good idea to move to the countryside.

I use the apostrophe to denote a step forward in time. But that time does not have to be of a fixed interval. It only serves to make actions distinct. In other words, whenever you feel the need to make a decision, that is when you step. Think of actions as course corrections and time as a record of the timestamp.

Inventory Control

Defining a problem in RL terms is a common issue for people who have a background in data science. Gaining more experience in the problem definition phase is worthwhile before you move on to other ways of solving MDPs.

Inventory control is a good example to start with because it is both useful and easy to understand. This example has direct analogies with pricing,

optimal stopping (selling an asset), maintenance, and more. I start with a very simple example and then expand it to a more complex version.

Imagine you own a small shop. This shop is so small you only have a single product. Each day customers buy your product so you need to restock. If you run out of stock, then you can't sell your product and make any money; you don't want to run out of stock. You rent your shop and it costs you a certain amount of money per square meter. You don't want to stock millions of items because you would need a larger premises. I could define many high-level problems, like the optimal size for the premises, or minimizing the delivery charges, but to start simply, I define the problem as deciding the best point to restock.

Let s represent the number of items in stock on a particular day. This is the state of the environment. In an attempt to keep things as simple as possible, I assume three possible states: no stock, one item in stock, or two items in stock, $\mathcal{S} \doteq \{0, 1, 2\}$ (the curly font here means that it represents a set).

Given the current state, the agent can perform an action. Keeping it simple, assume that the agent can take one of two actions. Either restock, which orders 1 new product, or do nothing: $\mathcal{A} \doteq \{\text{restock}, \text{none}\}$. If the current state was 1 and the agent performs the action, the next state will be 2. Also assume that you cannot order any stock when you are at full capacity.

The next thing missing from [Equation 2-4](#) is the transition probabilities. What are the probabilities that the agent shifts from one state to another? Since you are simulating the environment, you have to choose those, but they could come from observations of your shop. Say that the probability of a single sale during a day, $p(\text{sale})$, is 0.7. In the future you could predict the number of items sold. But for now, I use a Bernoulli distribution again. This means that in each state there is a 70% probability of a sale and a 30% probability of no sale.

NOTE

This is a purely hypothetical example. Of course it takes time to restock in real life. Simplifications are a necessary part of engineering, especially in early phases of development. Once you have a working prototype, then you can work on removing some of those simplifications.

The final thing you need is a reward, r . Your shop needs to make money, so you can reward yourself every time you sell a product. But sales are possible only if the product is in stock at the time, or if you restocked just in time. This means the reward is conditional on the current state and on a sale; no sale means no reward. You can see this definition in mathematical form in [Equation 2-5](#).

Equation 2-5. Inventory control reward definition

$$r \doteq \begin{cases} 1 & \text{if } s > 0 \text{ and a sale} \\ 1 & \text{if } a = \text{restock and a sale} \\ 0 & \text{otherwise} \end{cases}$$

Transition table

The next job is to define all the possible state transitions in the system and you can do this in three different ways. The first way is to present all possible combinations as a table, known as the *transition table*, shown in [Table 2-1](#).

Table 2-1. MDP transition table

s	a	s'	$p(s' s, a)$	r
0	none	0	$1 - p(\text{sale})$	0
0	none	0	$p(\text{sale})$	0
0	restock	1	$1 - p(\text{sale})$	0
0	restock	0	$p(\text{sale})$	1
1	none	1	$1 - p(\text{sale})$	0
1	none	0	$p(\text{sale})$	1
1	restock	2	$1 - p(\text{sale})$	0
1	restock	1	$p(\text{sale})$	1
2	none	2	$1 - p(\text{sale})$	0
2	none	1	$p(\text{sale})$	1
2	restock	2	$p(\text{sale})$	1

In [Table 2-1](#) there is a row for each possible combination of the current state, s , action, a , next state, s' , and probability of the transition into a state given a previous state and action, $p(s'|s, a)$; the reward for making that transition is in the final column, r . You can use this table to trace the *trajectory* of the agent.

NOTE

State 2 (two items) has only three possible transitions, because I have left off the final restock with no sale eventuality. I am saying that restocking when full is impossible.

Transition graph

Another way to represent this information is through a directed graph, known as a *transition graph*, shown in [Figure 2-5](#).

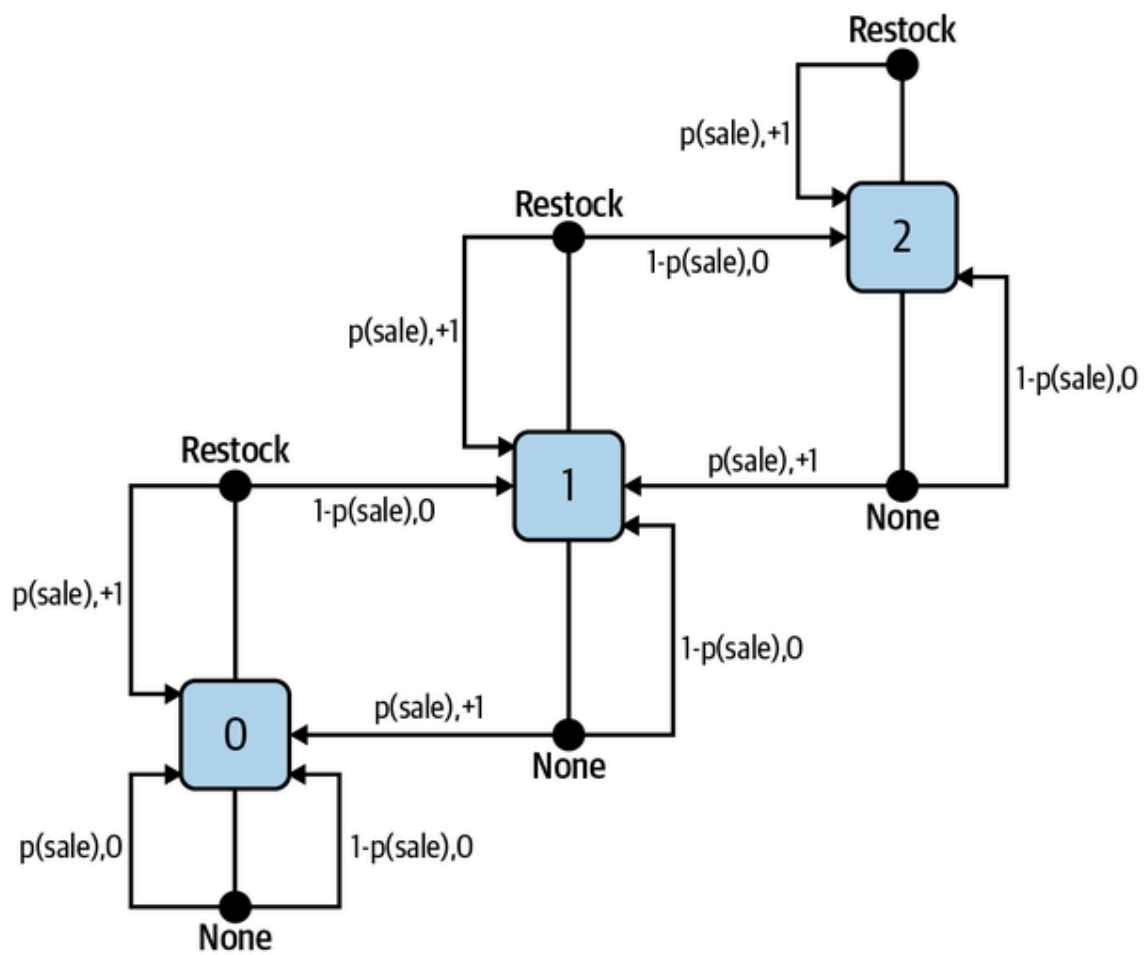


Figure 2-5. A graph representing the transition probabilities of the simple inventory problem.

The graph has two types of nodes (called vertices): one for the state, represented by a circle with the name of that state inside, and another for the action, represented by a solid circle. These connect through links (called edges), labeled with the probability of following that link after the action (the transition probability) and the reward obtained after following that link.

Transition matrix

The final way to represent the transition probabilities is with a matrix for each action, known as the *transition matrix*. You can construct the matrix in the same way as in [Table 2-1](#).

$$\begin{array}{l}
 p(s', a = \text{none}) = \begin{bmatrix} 1 & 0 & 0 \\ p(\text{sale}) & 1 - p(\text{sale}) & 0 \\ 0 & p(\text{sale}) & 1 - p(\text{sale}) \end{bmatrix} \\
 p(s', a = \text{restock}) = \begin{bmatrix} p(\text{sale}) & 1 - p(\text{sale}) & 0 \\ 0 & p(\text{sale}) & 1 - p(\text{sale}) \\ 0 & 0 & 1 \end{bmatrix}
 \end{array}$$

Each cell represents the probability of transitioning from the current state (0, 1, or 2 in rows) into the new state (0, 1, or 2 in columns). Note that

each row must sum to 1. For example, if the agent chooses not to restock, action $a = \text{none}$, you use the top transition matrix. If the shop has one item in stock, so the state is 1 (the second row), you can see that there is a probability of $p(\text{sale})$ of transitioning into state 0, you've sold the item, and $1 - p(\text{sale})$ of staying in the same state after no sale. Also note that you need a separate function or matrix for the obtained rewards as well.

TIP

I prefer the transition graph because it allows you to trace the trajectory taken by the agent with your finger. But you can only visualize the graph up to a small number of states and actions. Computational capacity is the only thing that limits the transition matrix, but it is difficult to interpret by eye.

Inventory Control Simulation

The previous sections presented all the ingredients required to build a simulation (the environment). But what about the agent? When should you restock?

You already placed a constraint that you cannot restock if the stockroom is full. That is one hard rule. But after that, what is the best action to take? If you look purely at the rewards, which represent the goal, then there is only a positive reward on making sales. Crucially, there is no negative reward for placing orders. In other words, according to the reward matrix, it doesn't cost anything to keep stock and it is free to order new stock. Given this reward structure, the best strategy is to keep ordering as much as you can.

Even though the strategy is obvious in this case, you should always test your idea—the scientific method. It is always a good idea to compare your theory against other baselines.

[Figure 2-6](#) compares three different restocking strategies, or *policies*.

Policies tell the agent which actions to take and in general the best ones maximize the total reward. The policies are keep restocking as much as possible, restock only when the stock levels drop to zero, or restock at random. The lines represent the cumulative rewards for each of the policies. The proportion of sales made is presented in brackets in the legend.

It is clear in this case that restocking at random is not a good idea, because quite often people wanted to buy but there was no stock. This was because the probability of a sale (0.7) was greater than the probability of restocking (0.5). The constant-restock policy managed to keep up with demand because you reordered at each time step. So the stock level (the state) bounced between 2 and 1. The reorder when zero stock policy bounced around the 0–1 level, because it only restocked when it hit zero. Effectively there is no performance difference between these two, but note that you might consider the keep buying policy to be wasteful, because you are using more space than you need, or safe, in case there's a rush. It depends on your high-level perspective and goal.

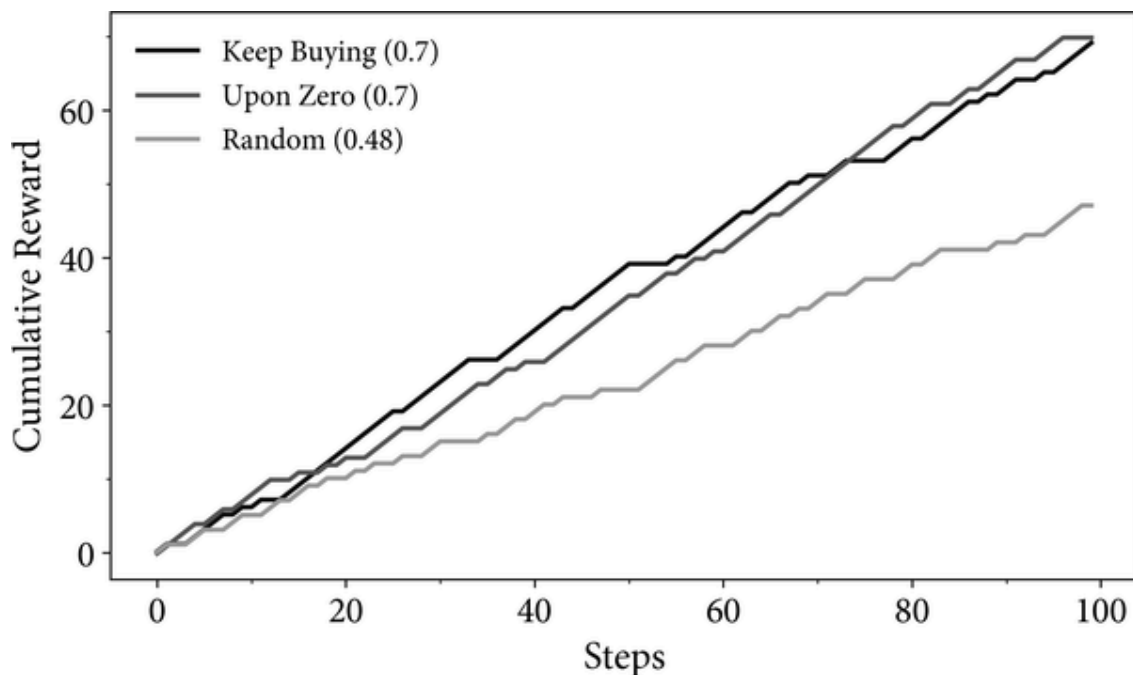


Figure 2-6. A comparison of different agent policies for the simple inventory problem. The proportion of sales made per day are in brackets. 0.7 is the maximum due to the problem definition.

TIP

This is an interesting example of how exactly the same data can mean different things to different people. Your boss might hate the waste, or they might love the buffer. Neither position is wrong.

You have just derived your first set of policies. And I haven't yet discussed how to learn an optimal policy—that will take the rest of the book. But this is an important result to take in; an optimal policy is dependent on all components of an MDP.

Changing the environment to include more stock, or allowing people to buy more than one item at a time, affects the best restocking strategy. Altering the reward function by introducing a penalty for ordering—a charge for delivery, for example—also alters the best strategy. In these cases it might be better to wait until you have zero stock then order a full 2 units worth of product. The reward function defines what problem you want solved, not how to achieve it.

Standard MDPs require the state to be fully observable. If the agent cannot observe the true state of the system, then how can it pick an optimal action? There is an extension to the MDP model called partially observable MDPs (POMDPs). Here, you can introduce an extra mapping between the observations and the actual state. In other words, the agent would need to learn how to map actions to observations that are mapped to states that it cannot observe, a double mapping. As you can imagine, this is tricky to solve, because of more unknowns. In many applications, engineers trick the system back into a standard MDP by feeding more information from previous time steps into the current time step. Knowledge of the past can help the agent figure out how the actions altered the perceived states over a longer period of time.

I also want to stress that the important thing here is the observation of the state, not the environment. Agents do not concern themselves about what is happening behind the MDP interface. You can alter the simulation as much as you want, so long as you do not change the interface. In the inventory example, if you altered the customer's buying behavior, it only affects the optimal policy. It does not change *how* the agent learns.

Bear these points in mind when you read the rest of this book.

Policies and Value Functions

A policy is like a strategy. For example, you can't be entirely sure what the other football team is going to do (the other team's activities are stochastic), but you can approximate their actions and tell your players to act accordingly. A policy is a mapping from states to potential actions. Precisely how you do that mapping is the major difference between a large number of RL algorithms. But why should you build a mapping in the first place? And how can you evaluate different policies?

Discounted Rewards

In many problems, including the inventory and website button examples, the reward at each step is straightforward to reason about. At each time step the agent may or may not receive a reward. Through multiple steps (in the inventory example), the agent may receive multiple rewards. In general the challenge is to maximize the total reward that you expect to receive.

The *return*, G , is the total reward from this current step up to the final time step (or infinity). The first dot-equals sign can be read as “I define G to be.” Note that the agent receives the reward after transitioning into the next step. Most practical implementations return the state and the reward at the same time, so they set the initial reward to zero.

[Equation 2-6](#) sums all the future rewards up until a final time step, T . This final state is called a *terminal state* and only occurs in *episodic* tasks. An episode represents one full run through all steps that end at a natural termination point. This could be when your robot falls off a cliff, or if your character in a game dies. But of course, there may be tasks that continue forever, like the inventory example—if you allow me to be optimistic about the state of the retail market. In this case the terminal state is $T = \infty$ and the expected return could also be infinite.

Equation 2-6. Return

$$G \doteq r + r' + \dots + r_T$$

To mitigate against the explosive power of infinity, [Equation 2-7](#) adds a discounting factor that exponentially reduces future rewards.

Equation 2-7. Discounted return

$$G \doteq r + \gamma r' + \gamma^2 r'' + \dots = \sum_{k=0}^T \gamma^k r_k$$

In [Equation 2-7](#), T represents the terminal state and could be infinity. γ is called the *discount rate* or *factor*. It controls how soon future rewards are ignored and should be a value between 0 and 1, inclusive. In other words, if $\gamma = 0$ then only the present reward is taken into consideration. If $\gamma = 1$ then this is the same as [Equation 2-6](#). Typically you want to take the future rewards into account to be able to solve problems where the

reward is far into the future. But precise values will differ depending on the problem at hand. In most nontrivial examples the discount rate is set between 0.9 and 0.99.

Therefore, you want to build an agent that can generate the largest *expected return*. One question you might have is how to calculate the expected return, given that it depends on future rewards. The answer is through iteration and learning which states and actions map to the best rewards. In other words, you should iterate to find the best policy.

Predicting Rewards with the State-Value Function

In the inventory example of [“Inventory Control Simulation”](#), I arbitrarily chose three restocking strategies. But I want to optimize the choice of action to maximize the expected return automatically. This is the purpose of the *policy*; it maps states to actions. Formally, it is the probability of choosing an action given a state, $p(a|s)$, and is denoted by π .

TIP

Policies can be improved by nudging the gradient of the policy with respect to the expected return. I will return to this in [Chapter 5](#).

You can calculate the expected return from the return, G , expected from your current policy, π , starting from the current state, s . This is called the *state-value function* for the policy and is shown in [Equation 2-8](#).

Equation 2-8. State-value function

$$V_{\pi}(s) \doteq \mathbb{E}_{\pi}[G|s] = \mathbb{E}_{\pi}g \left[\sum_{k=0}^T \gamma^k r_k \middle| sg \right]$$

I appreciate on first glance that this equation looks complex, so let me walk you through it. There are three parts to the equation, separated by the equals signs. On the left, V is the symbol used to denote the value function.

TIP

For those who aren't math-native, I find that imagining the mathematical functions as functions in software helps.

The second part, the definition of the function, is saying that the value function is the expected value of the return, given a current state, following the policy π . The symbol $\mathbb{E}$ is the expectation operator (more about this in a moment). This instructs you to calculate the expected value of the term in the brackets. And the π subscript tells you that the agent should follow the policy to calculate that expectation.

The third part of the equation on the right is an expansion of the term in the brackets. You know how G is defined from [Equation 2-7](#). And you know you need to start from the current state. At this point the math inside the brackets is basically saying “sum all the rewards from this point forward.”

[Equation 2-8](#) has two important points. The first is the expectation operator. Most RL algorithms assume rewards are characterized by a normal distribution and the expectation can be calculated as the sample mean. In other words, the result of the expectation operator is the average discounted reward from that given state. Rewards in some applications may not be normally distributed. For example, what would you choose: one single big reward or many small rewards? In both situations the mean is the same, but the distribution of rewards isn't. Reward nuances can be hidden by the expectation operator. In fact, a lot of research has recently focused on trying to evaluate and use the distribution of rewards instead (see [“Distributional RL”](#)).

The second important part is the policy. You can only calculate the expected return if you already have a policy. But this equation is meant to help you quantify the performance of a policy. This is a classic chicken and egg problem that RL solves by starting randomly and iterating to find an improvement.

Simulation using the state-value function

I find that my intuition of an equation is improved through a simple simulation. I developed a program that simulates positions on five squares

(the states). The agent starts on the lefthand side. In each time step the agent can move left or right. The simulation ends if the agent reaches the goal state of square 5, on the righthand side, or if the agent moves left on square 0, off an imaginary cliff. To keep things as simple as possible, the policy is a random one, in which it randomly chooses either left or right on every time step.

TIP

You should continuously consider different environments. *Overfitting* is the process of accidentally building a model or an algorithm that works with very specific data or circumstances. It is easy to improve an algorithm to the point where it overfits your simulation and find it does not work in real life, for example. Industrial RL algorithms should be robust, and considering different environments is one way to help your algorithm *generalize*. Also, using different environments is an important way of gaining experience. Hence, I intentionally keep switching environments throughout these examples.

The goal of [Equation 2-8](#) is to calculate the expected reward for each state. You can do this by recording the position of the agent through each *epoch*, which is a continuous loop of the environment until the agent reaches a terminal state. Eventually, you will obtain a reward and you can go back and look at all the states visited along the way. You then add the reward to a cumulative sum and keep a counter to calculate the average return observed after visiting all of those states.

TIP

Whenever you perform an experiment, it is prudent to imagine some theoretical expectations, or priors—defining a hypothesis is the heart of the scientific method.

Imagine you are on square 1, on the lefthand side. Imagine how to get to each of the terminal states. You can move one square to the left and fall off the cliff. Or you can move four squares to the right and reach the end. Your first policy is a random one; it randomly chooses a new direction uniformly. It randomly picks to move one square left more often than four squares right. On average, only 1 out of 5 attempts reach the righthand side; the rest fall off the cliff. If you set the the reward to 5 (to match

the number of squares) and you do not penalize for falling off the board, then the expected reward of each state should be the same as the position of the square.

In this example, the expected return from square 1 should be 1. The expected reward in the terminal state should be 5, because there is no need to move. In intermediary states, because the agent is moving randomly, there is still a chance that the agent might randomly move back toward the cliff. So even when on the fourth square, one step away from the reward, the random agent 1 out of 5 times will stumble back to the cliff, on average.

If this doesn't make sense, imagine two squares where a single move to the right reaches the goal state. Now increase the number of squares.

Note that in this example you are setting the discount factor to 1. In more complicated environments you should set this to less than 1 because early moves in an episode are typically less important than those close to a terminating state. But the precise value depends on the reward and whether the problem is continuous or not.

When I run this simulation it takes a large number of iterations to stabilize. After the first iteration, the agent jumped immediately to the left and died. So the expected return at iteration 0 is `[0.00, NaN, NaN, NaN]`, where positions in this array represent the squares 1 through 4. Square 5 isn't recorded because there is no action to take in the terminal state.

After ten iterations the agent has reached the goal state a couple of times: `[0.77, 3.00, 5.00, 5.00]`. After a thousand iterations the expected returns begin to get close to the previous estimate: `[1.04, 2.09, 2.93, 3.83]`. After ten thousand random experiments the result is very close: `[1.00, 1.97, 2.92, 3.92]`. The more iterations I run the closer it will get to the true expected value. The result of this simulation can be seen in [Figure 2-7](#).



Figure 2-7. The result of simulating the calculation of the state-value function in a simple environment. Each square represents a state. The value inside each square represents the expected return starting from that state and following the random policy. After repeated iterations the expected return from each state approaches the theoretical expectation.

Another interesting experiment to try is to change the starting position. After ten thousand iterations the results are the same (within the range of random fluctuations). You could try adding discounting or penalize for falling off the cliff to see what happens. I encourage you to develop your own code to gain intuition, but of course you can refer to the practical examples on the accompanying website (see [“Supplementary Materials”](#)).

Predicting Rewards with the Action-Value Function

The previous example shows you how to predict the expected return from a given state, which is the average reward observed after previous visits to that state. But actions are not all created equal. What if taking one action led you closer toward rewarding states? Many algorithms use this information to help decide which is the best action to choose. This requires an expected return for any action in any state and is shown in [Equation 2-9](#).

Equation 2-9. Action-value function

$$Q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G|s, a] = \mathbb{E}_{\pi}g \left[\sum_{k=0}^T \gamma^k r_k \middle| s, a, g \right]$$

[Equation 2-9](#) is the *action-value function* for a policy π . You can see that it is virtually the same as [Equation 2-8](#) except for the fact that there are two parameters to the function: the state and the action. You can imagine this being implemented as a two-dimensional lookup table to store the rewards for states and their actions. This function is often called the Q-function, for brevity.

[Figure 2-8](#) shows the result of the same simulation from [“Simulation using the state-value function”](#), but with the action-value function instead. [Equation 2-9](#) brings the addition of storing the expected return for both the state and action. One result is that moving left from square 0 will always return a reward of zero. Moving right from square 4 will always result in a reward of 5.0.

Another interesting result is that the expected value for all the right actions are higher than the corresponding left actions. This is because the further the agent moves to the right, the better the chances of actually reaching the goal state. To finish, let me ask you a question. Given these states, actions, and expected returns, can you think of a policy, or a strategy, that is optimal for this environment? Can you imagine yourself as an agent, making a decision at each square? How do you choose which direction to take?

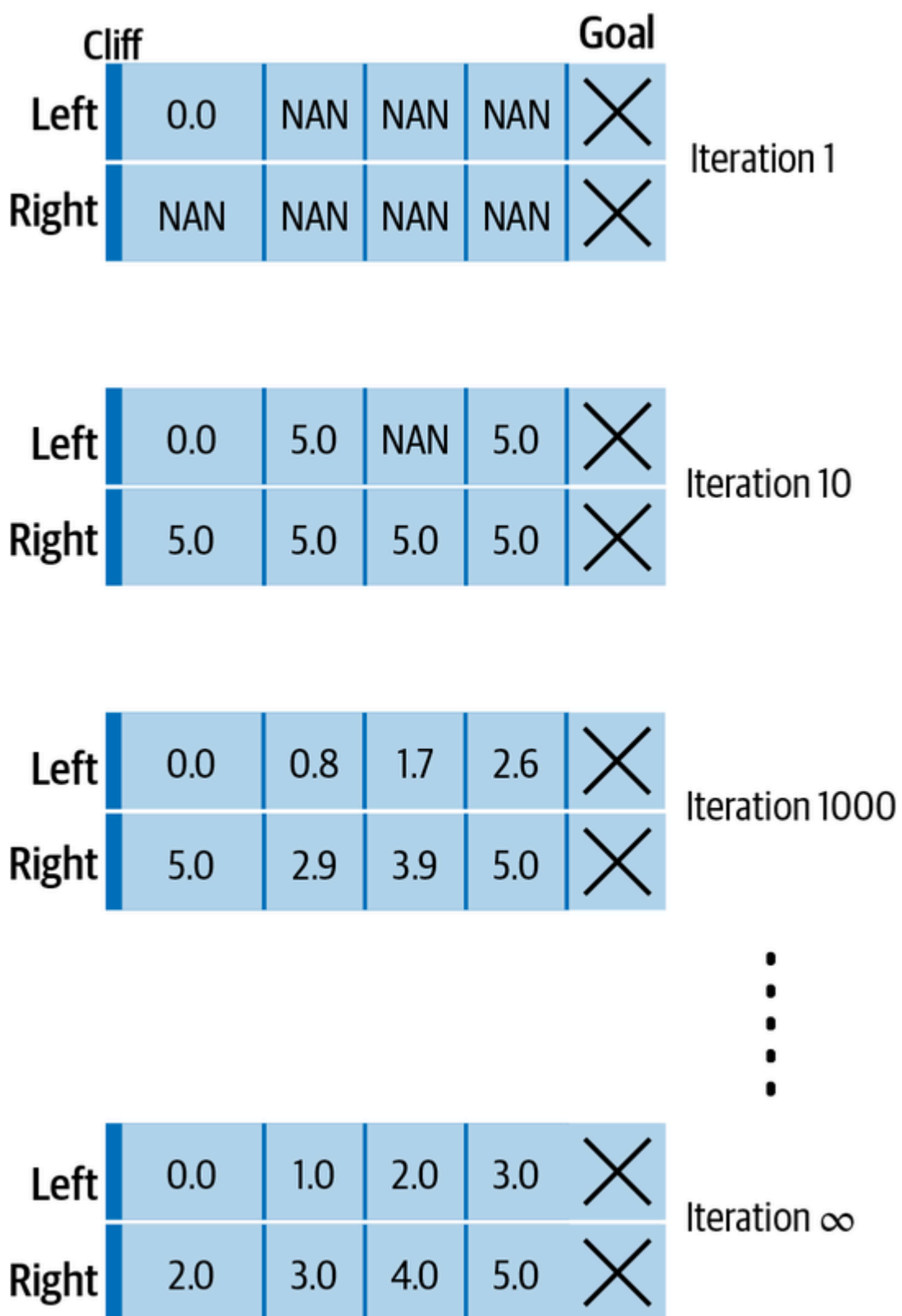


Figure 2-8. The result of simulating the calculation of the action-value function in a simple environment. The setup is the same as in [Figure 2-7](#). The difference here is that I am recording the result of the action as well as the state.

Optimal Policies

An optimal policy is one that when followed generates the highest possible expected return. This means that there is at least one policy that provides the greatest return, which I denote as π^* . The equivalent optimal state-value and action-value functions are $V^*(s)$ and $Q^*(s, a)$, respectively.

The state-value functions and the action-value functions are related. They both represent the same information but at different levels of granularity. The state-value function is the expected reward averaged over all actions, which provides a low-resolution view of which states valued highly. The action-value function provides a high-resolution picture of which actions and states are the best.

Imagine you have good estimates for the expected values, because you have been randomly moving around the environment for a long time, for example. Then an optimal policy would suggest actions that move the agent toward higher expected returns. In other words, an optimal policy is one that chooses the action with the maximum return.

TIP

Most RL problems are framed to maximize the return, but this isn't strictly necessary. I recommend sticking to maximization though, to avoid confusion.

Now that you have an optimal policy, imagine calculating the state-value and action-value functions again, when following this policy. Both values will be exactly the same. The state-value function is equal to picking the best action in the action-value function. [Equation 2-10](#) represents this idea in mathematical form and is called the *Bellman optimality equation*.

NOTE

The official Bellman optimality equation is more formal, aggregating over the transition probabilities and the probability of a policy selecting an action. For more information see [“Further Reading”](#).

Equation 2-10. Optimal value function

$$\begin{aligned} V^*(s) &\doteq \operatorname{argmax}_{a_s \in \mathcal{A}(s)} Q_{\pi^*}(s, a_s) \\ &= \operatorname{argmax}_{a_s \in \mathcal{A}(s)} \mathbb{E}_{\pi^*} g[G|s, a_s] \end{aligned}$$

[Equation 2-10](#) reuses the definition of the action-value function, Q , from [Equation 2-9](#), using the return G . It states that choosing the action that maximizes the action-value function at every step is the same as the opti-

mal value function. Let me restate that, because it is important. You obtain the highest total reward if you repeatedly choose the action with the highest expected return.

This might not make sense at first, because you can imagine that there may be greater rewards elsewhere. But remember that equations have full knowledge about all future rewards (look at the return, G , in the equations). So the action-value function (or state-value function) has already accounted for the fact that the agent is going to visit that rewarding state in its estimate.

So to answer the question from the end of [“Predicting Rewards with the Action-Value Function”](#), the optimal policy is one that repeatedly chooses the action with the highest reward. In that example, the action with the highest reward was always on the righthand side, so the optimal policy is to choose “right” every time.

Monte Carlo Policy Generation

The example in [“Simulation using the state-value function”](#) randomly selected actions and recorded the reward, but you don’t have to. Instead you can exhaustively search every possible permutation of moves and store the trajectory. Then the optimal policy is the trajectory that achieved the highest reward.

But this strategy is impractical in most cases. Even for the simple example in [Figure 2-7](#), you could imagine a strategy that repeatedly picks right then left and never terminates. You could help the agent by constraining the exhaustive search, but in general, you need to use something else.

Formally, the issue is the high dimensionality of the problem. The go-to algorithm that happily digests high-dimensional sampling problems is *Monte Carlo* (MC). Imagine a black-box environment, illustrated in [Figure 2-9](#), in which you have no visibility. You control the actions presented to the environment, so you know the distribution of the inputs. But you have no idea of the distribution of the outputs, because the environment is not under your control. Characterizing the output distribution helps you pick the action that delivers the highest return (on average).

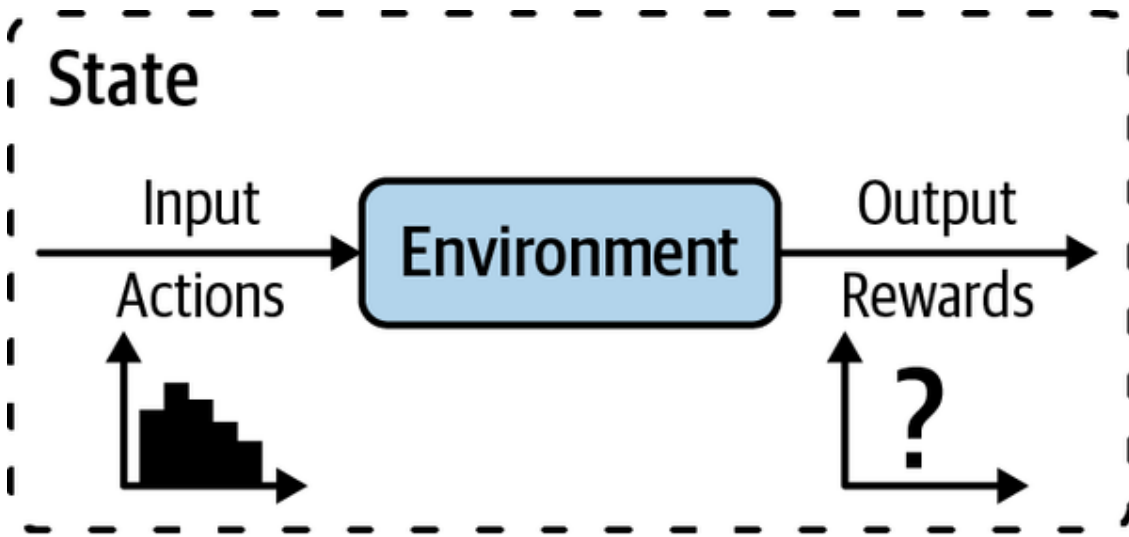


Figure 2-9. A black-box description of the Monte Carlo technique. Inputs are known, outputs are not. By randomly stimulating the inputs you can observe and characterize the output.

MC techniques randomly sample the input to observe what happens at the output. After adequately sampling actions you can characterize the output and therefore determine the reward. The term is used broadly to include any estimation method that includes a significant random component. But in RL it denotes techniques that average the reward returned over entire trajectories. You can find out more about MC techniques in [“Further Reading”](#).

Now I can turn this idea into an algorithm, one that first generates an entire trajectory then updates the action-value function according to the observed reward. Given time the policy will tend toward the optimal one.

[Algorithm 2-3](#) is called an on-policy MC algorithm because it (a) samples full trajectories and (b) is estimating the value of the policy and using it to select actions at the same time.

Algorithm 2-3. On-policy MC algorithm

- 1: **input:** a policy function, $\pi(a|s, Q_\pi(s, a))$
- 2: Initialize $Q(s, a) \leftarrow 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$,
 $Returns(s, a) \leftarrow []$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- 3: **loop:**
- 4: Generate full episode trajectory following π
- 5: Initialize $G \leftarrow 0$
- 6: **loop** for each step of episode, $t \doteq T - 1, T - 2, \dots, t_0$:
- 7: $G \leftarrow \gamma G + r$
- 8: **if** (s, a) not in $(s_0, a_0), (s_1, a_1), \dots, (s_{t-1}, a_{t-1})$:
- 9: Append G to $Returns(s, a)$
- 10: $Q(s, a) \leftarrow average(Returns(s, a))$

[Algorithm 2-3](#) begins in step (1) by passing in an implementation of a policy. This algorithm is typically presented with the policy built in, but I find it simpler to ignore the details of the policy and instead inject it as a dependency. The policy, $\pi(a|s, Q_\pi(s, a))$, returns an action for a given state using the action-value function populated in this algorithm. Step (2) initializes the action-value function and creates a buffer for all the interactions with the environment.

Step (4) generates a trajectory for a full episode using the current policy. In my code I created a policy that picks the action with the highest expected return, like in [Equation 2-10](#), with an ϵ -greedy search.

Step (6) iterates over every step in the episode in reverse. This is an important distinction; the agent chooses actions based upon *future* rewards. So step (6) starts at the end and iterates backward, summing rewards as it goes. Eventually it will end up at the first state and then you have one prediction of the expected return from that state.

Step (7) adds the reward discounting to the expected return from [Equation 2-7](#). If the current state-action pair was not visited earlier in the trajectory [step (8)] then step (9) appends the pair to the buffer. This *first-visit* implementation estimates $V_\pi(s)$ as the average of the returns from the first visit to the state. *Every-visit* implementations estimate $V_\pi(s)$ as the average returns from every visit to a state. Both implementations lead to the same estimate but researchers have studied the first-visit algorithm in more depth and found it slightly less erroneous (every-visit action value estimates are more noisy—see [“Further Reading”](#) for more information).

Finally, in step (10), the current action-value prediction is updated. This provides the policy with an estimate of the expected return for any state-action pair. This estimate is the average discounted return observed over all trajectories. Using an average makes the assumption that the expected return is normally distributed; it may not be, but this assumption works well in practice.

The results from [Algorithm 2-3](#) are the same as in [Figure 2-8](#) if you set the ϵ -greedy parameter to explore-only and the discount factor to 1.

Value Iteration with Dynamic Programming

In the previous sections I propagated the reward backward through a trajectory at the end of an episode. But there is another way.

Imagine an algorithm that scans one step ahead. Given a state, the agent can peek one step ahead to observe the next expected return. In environments where the terminal state provides the reward, like in [Figure 2-10](#), in iteration (1) the agent will look one state ahead and see the terminal state, so it knows which action to take. In iteration (2) the agent sees the state before the terminal state, and knows which action to take again. This continues until the policy stabilizes.

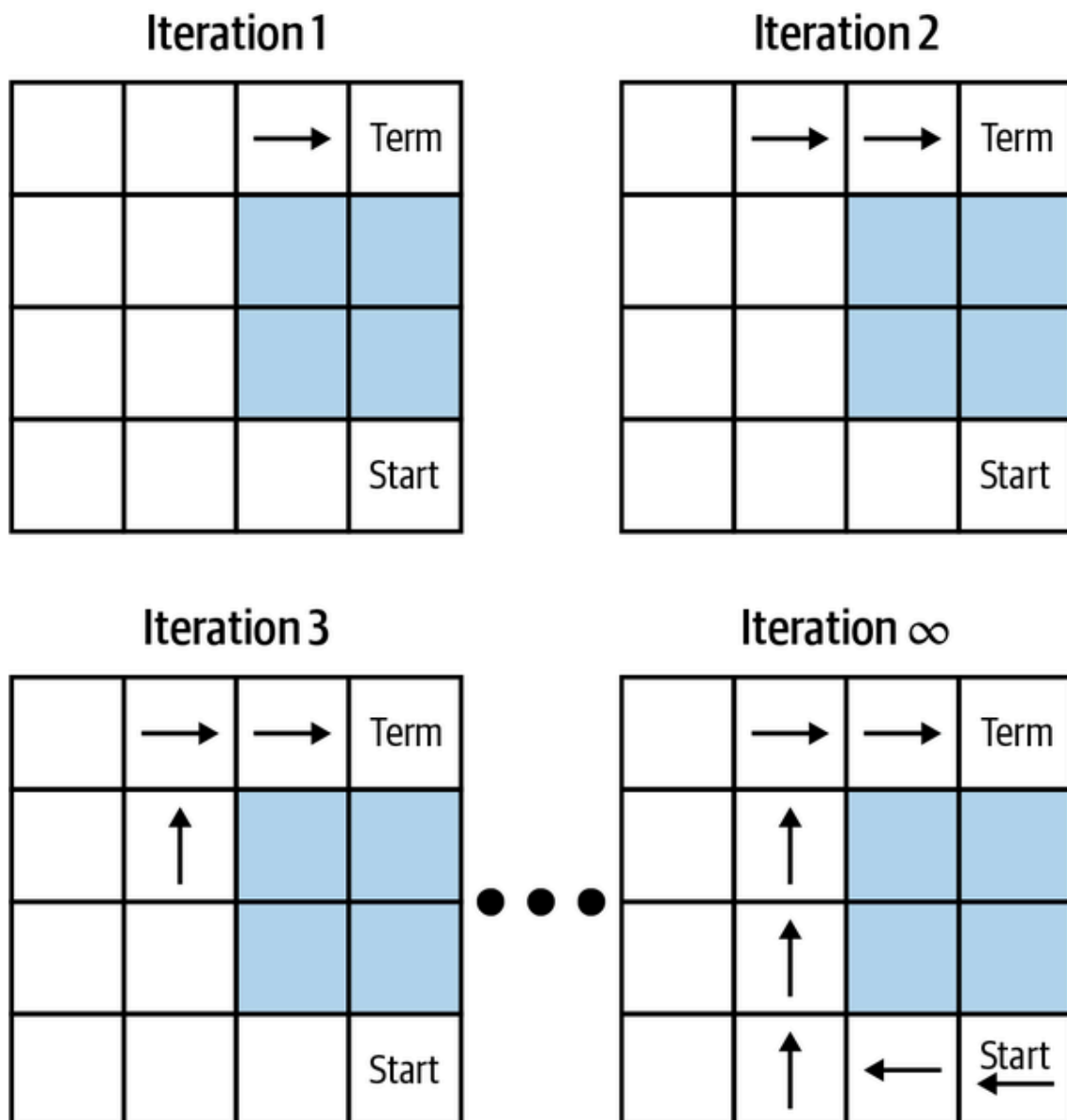


Figure 2-10. The policy generated when looking one state ahead over different iterations of the algorithm.

The literature often creates two abstractions based upon this process: policy evaluation and policy improvement. *Policy evaluation* provides the estimate of the expected value. *Policy improvement* updates the policy to take advantage of new knowledge coming from the value function, like a new promising path to explore.

When you combine evaluation and improvement, this is called *generalized policy iteration* (GPI). Implementations differ depending on at what level you interleave improvement and evaluation. For example, you could wait until the end of an episode to perform an improvement using MC methods, like in [“Monte Carlo Policy Generation”](#).

Alternatively, you could update your value estimates after each step. *Dynamic programming* methods look one step ahead and iterate over all actions. Contrast this with MC techniques that choose a single action, but have to simulate a full episode.

[Figure 2-11](#) illustrates the difference through something called a *backup diagram* for the state-value function. Open circles represent states and filled circles represent actions. In DP, in a given state, it searches across all possible actions. This makes it possible to update the policy after a single step. MC first generates a full episode by following a policy, then it updates the policy.

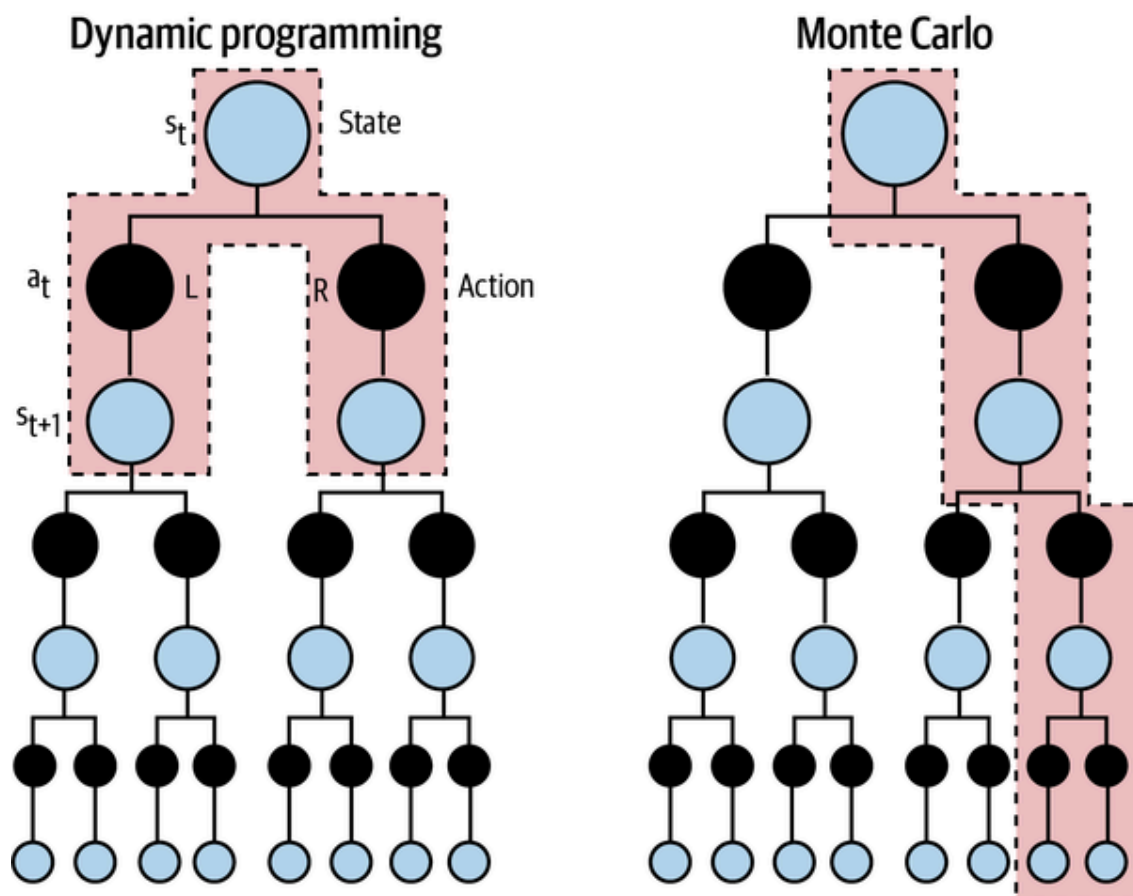


Figure 2-11. A backup diagram showing the difference between dynamic programming and Monte Carlo policy update techniques (for the value-state function).

Implementing Value Iteration

A technique called *value iteration* implements DP, but there is a problem. Value iteration depends on knowing the transition probabilities in full; you need a model of your environment. This is because the agent needs to know which actions it is allowed to try. In a two-dimensional grid, for example, you need to tell the agent that it cannot move outside of the boundaries by setting those probabilities to zero.

If you have control over the design of the environment and the problem is simple enough, then you can specify the transition probabilities in advance, like in the examples of [“Inventory Control”](#) or [“Simulation using the state-value function”](#). [Chapter 3](#) develops generic algorithms for use when you don’t have a full model of the environment. For now, assume you do and you can pass it into [Algorithm 2-4](#).

Algorithm 2-4. Value iteration for generating an optimal policy

```
1: input: transition probabilities,  $p(s', r | s, a)$ , stopping threshold,  $\theta > 0$ 
2: Initialize  $V(s)$ , for all  $s \in \mathcal{S}$ 
3: do :
3a:    $\nabla \leftarrow 0$ 
4:   loop for each  $s \in \mathcal{S}$ :
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ 
7:      $\nabla \leftarrow \max(\nabla, |v - V(s)|)$ 
8:   while  $\nabla > \theta$ 
9: output: an optimal policy,  $\pi(s) \doteq \operatorname{argmax}_{a_s \in \mathcal{A}(s)} \sum_{s', r} p(s', r | s, a_s) [r + \gamma V(s')]$ 
```

In step (1) of [Algorithm 2-4](#), you need to specify the transition probabilities and a stopping threshold. In a toy grid environment, for example, the transition probabilities are equal for all actions except for those that step off the grid or are blocked by a wall, which have a transition probability of zero. The stopping threshold controls how long to keep refining the value estimate.

Step (2) initializes the state-value buffer for all states and a variable for maintaining the current amount of error. Step (3) enters a do-while loop and step (4) loops over every state in the environment. Step (5) stores a local copy of the state-value estimate for the current state for use later.

Step (6) is where the action happens. Starting from the lefthand side, it finds the largest expected return after trying all actions. The sum is summing over all potential next states and rewards for an action (because even though you select action left, you might end up going straight on). Then the probability represents the chances of landing in a next state with a reward given a current state and action. And finally the value of this series of events is the reward from the action plus the value of the next state.

I recommend that you reread the equation and the description again because step (6) squeezes a lot into one line. Now that you have reread it, let me give you a hand-worked example that I visualize in [Figure 2-12](#).

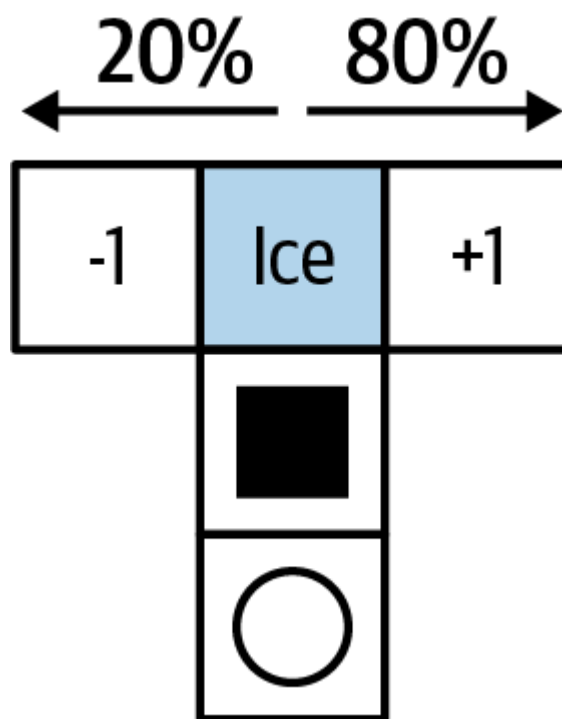


Figure 2-12. A stylized example demonstrating the value iteration update. The square represents your current position. The values represent the rewards. The expected value for the current state is 0.6.

Imagine you are standing in a grid, represented by the square, where you can only move forward or backward and all state-value estimates are zero. If you step back the movement is certain but you receive a reward of 0. If you step forward you will slip on ice and there is a 20% chance that you will fall to the left with a value of -1 and an 80% chance that you fall to the right with a value of 1.

Step (6) from [Algorithm 2-4](#) aims to find the highest value for any action. This environment has two actions: up and down. The probability of transitioning down when actioning down is 1. The reward is 0 and since all the value estimates are zero the total sum for the down action is $1 \times 0 = 0$. For the up action, the two outcomes have two rewards. This sum becomes $0.2 \times -1 + 0.8 \times 1 = 0.6$. Hence, the maximum value over all actions for the current state is 0.6.

The previous example was for a single state, but that calculation depends on other states. This is why [Algorithm 2-4](#) needs to iterate over all states to fill in the gaps. After the first pass of all states, it is likely that you need to update them all again because the values have changed. Step (7) records the maximum error to allow step (8) to decide whether to keep iterating. If you are interested, you can look up the convergence proof from the references in [“Further Reading”](#).

Finally, in step (9), the algorithm outputs a deterministic policy that chooses the action with the highest expected return.

Results of Value Iteration

The results of an implementation of [Algorithm 2-4](#) are shown in [Figure 2-13](#). A discount factor of 0.9 was used to find the optimal policy for the simple cliff problem encountered earlier.

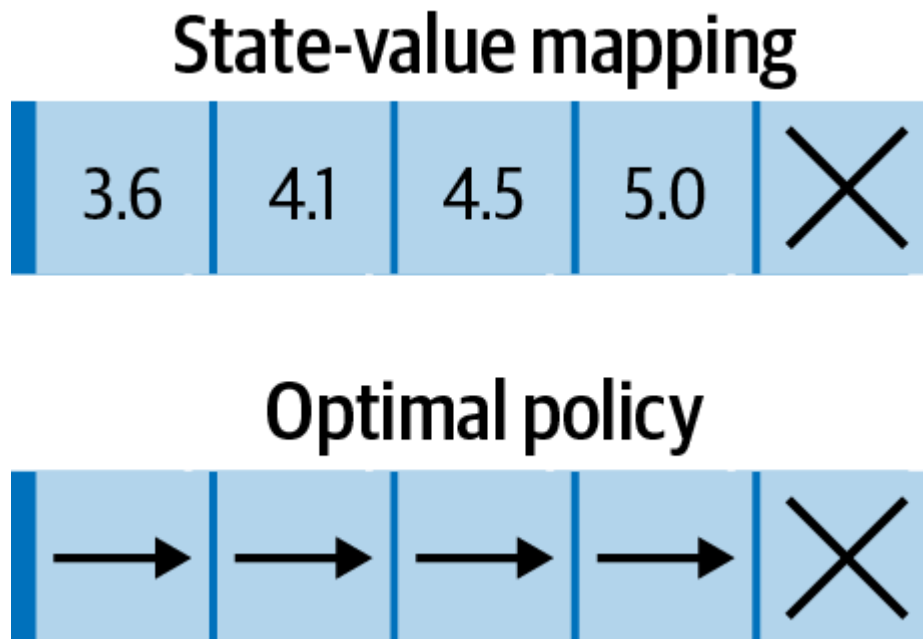


Figure 2-13. The result of training an agent using value iteration with a discount factor of 0.9. This image shows the state-value estimates and the optimal policy.

Like the MC version, the state-value map is simple to interpret. The closer the agent is to the goal, the greater the expected return. The slight difference is that the value is not dependent on randomly reaching the goal state. It is a discounted version of the reward if you were to follow a policy. The resulting policy is the same, however. And I could have used the same policy evaluation technique in the MC algorithm, too.

If I were to insert [Algorithm 2-4](#) (the policy improvement algorithm) into an agent's interaction loop (policy evaluation), then you could improve the policy on every time step. Contrast this to MC techniques that have to wait until the end of an episode. DP can improve a policy much faster than MC, which is very important when dealing with problems with long (or infinite) episodes.

But four caveats prevent you from using DP in all but the simplest of environments. The first thing to watch out for is that you have to use dis-

counting. If you didn't, all states would have a value equal to the reward. You must use discounting to direct the agent and suppress long trajectories. This is a common theme throughout all RL: prefer shorter trajectories.

Secondly, [Algorithm 2-4](#) finishes when the agent has updated all states. That is fine in the toy examples seen so far, because they all have few states. But once the number of states or actions increases, then it becomes computationally prohibitive to use DP.

A third point, related to the previous, is the *sampling efficiency* of an algorithm. DP needs to visit all states and actions. This requires at least as many samples from the environment as states multiplied by actions. This might be a problem for environments you cannot simulate. Other model-based RL algorithms may be more efficient because they can interpolate a policy with fewer simulations. MC techniques are also more sample efficient because it is likely that they will not have to visit every single state to find a region of states that produce the best return.

Finally, and probably the biggest problem of all, is that you need the transition probabilities. You either need a model of the dynamics of the environment or you can learn a model via sampling methods. If you perform an action multiple times, you can estimate the probability of landing in any states. You can look forward to this in the next chapter.

Summary

This chapter began by demonstrating that bandit testing, something that you might have used before, is a form of RL. It is limited though, because most industrial applications have many more states and delayed rewards.

Markov decision processes (MDPs) are the framework upon which all problems are built. You can reframe many problems to fit into the MDP paradigm. The key elements of an MDP are an environment that exposes an observation of its state, a reward to denote success or failure, and actions that mutate the internal state of the environment.

By calculating the expected value of next states, algorithms can choose an action that maximizes the total reward. You saw this in the inventory con-

trol simulation. I constrained the problem to make it simpler, but you can find examples of more complex scenarios in the literature.

An optimal policy is something that the agent can follow to obtain a high reward. Finding the optimal policy is difficult and the algorithms presented in this book try to achieve this in different ways.

Finally, you discovered two ways of searching through an environment's states to estimate the return: Monte Carlo (MC) and dynamic programming (DP). MC methods work well for small state spaces and when learning performance isn't important. But DP is more efficient in the sense that it can learn prospective policies immediately, as opposed to MC, which has to wait until the end of an episode.

The next chapter expands upon DP and presents one of the most common RL algorithms used in industry today.

Further Reading

- Statistics:
 - Downey, Allen. 2014. *Think Stats: Exploratory Data Analysis*. O'Reilly Media.
- A/B testing and bandit algorithms:
 - White, John. 2012. *Bandit Algorithms for Website Optimization*. O'Reilly Media.
 - [My Python implementation of the Bandit simulation library](#).
 - A thorough tutorial on sampling methods for bandits.¹
- MDPs:
 - Puterman, Martin L. 2014. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons.
- Bellman equations, DP and MC:
 - Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- MC techniques in other domains:
 - Hilpisch, Yves. 2018. *Python for Finance: Mastering Data-Driven Finance*. O'Reilly Media.

- ¹ Russo, Daniel, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. 2020. [“A Tutorial on Thompson Sampling”](#). ArXiv:1707.02038, July.

Table of contents collapsed